

Test Case Lab — Instrument, Measure, Fix (One Disciplined Sprint)

Branch: `feat/generation-quality-engine`

Scope: Instrument the existing generation pipeline, measure it against a sealed senior-QA benchmark, find the single earliest stage where coverage is lost, fix **only** that stage, and prove no regression. No new engines. No new architecture.

1. The discipline (what this sprint did and did not do)

This was **not** a brainstorming or engine-building sprint. It followed one method, end to end:

1. **Seal a gold benchmark first.** Five requirements (Create Employee, Login, Checkout, Leave Request, Password Reset) were specified as expected validations from QA expertise **before** looking at a single line of generator output. The expectations were never edited afterwards to match what the tool produced.
2. **Instrument the live pipeline.** A diagnostic harness runs each requirement through every stage and dumps INPUT / OUTPUT / what each stage ADDED / what it LOST.
3. **Measure the gap** between final output and the sealed benchmark.
4. **Locate the earliest stage** where each missing validation first disappears.
5. **Fix exactly one stage** — the earliest, highest-leverage one — and nothing else.
6. **Regression-test all five** benchmarks after the fix.

Honesty caveat, stated up front. The harness exercises the deterministic portion of the live pipeline — classification → scenario planning → plan block → quality report. It does **not** call the LLM write-up or the database. The LLM is explicitly instructed not to invent and not to drop scenarios, so the deterministic plan is the **coverage ceiling** the LLM is bound to. Every percentage below is that ceiling, produced by the harness and traceable to `scripts/audit-output/*.json`. No number here is estimated.

2. What the pipeline actually is (measured, not assumed)

The live path behind `POST /api/test-coverage/generate` is:

```
classifyQACategory → planScenarios (QA_KNOWLEDGE_BASE) → buildScenarioPlanBlock
→ LLM write-up → dedup → buildQualityReport
```

Two previously-built folders — `src/requirement-understanding/` and `src/validation-planning/` — were confirmed **dead code**: nothing outside their own directories imports them. The harness runs them for comparison and marks them `[NOT WIRED TO LIVE GEN]`. They are not what produces test cases today.

3. The measurement — coverage vs the sealed benchmark

Coverage is scored on the design dimensions the benchmark cares about — Business Rules, Validation Types, Workflow, Permission/Authorization, Security, and Missing-Validation families — **not** on Positive/Negative/Edge counts. P/N/E is an outcome of good design, not a design target.

Numbers are “full coverage mode” (all validation families enabled), using one identical matcher for both columns so the comparison is apples-to-apples.

Requirement	Classified as	Before fix	After fix	Critical gaps (before → after)
Create Employee	crud ✓	18/26 — 69%	24/26 — 92%	1 → 0
Login	authentication ✓	18/18 — 100%	18/18 — 100%	0 → 0 (no regression)
Checkout	checkout ✓	17/29 — 59%	24/29 — 83%	1 → 0
Leave Request	crud ✗ (want workflow)	13/25 — 52%	15/25 — 60%	2 → 1 (see §6)
Password Reset	authentication ✓	17/21 — 81%	20/21 — 95%	1 → 0

Critical-severity gaps went from 5 to 1. The one survivor is a classification problem, not a knowledge problem — documented in §6 as the next earliest stage, deliberately left for the next iteration.

4. The diagnosis — where was coverage lost?

Per-validation, the harness marks the **earliest** stage where each missing item first disappears:

- **A — Classification:** routed to the wrong category, so the wrong knowledge is consulted.
- **B — Scenario Planner / KB depth:** correct category, but the knowledge base has no such obligation.
- **C — Coverage selection:** the KB has the obligation, but the default (positive-only) mode never emits it.

Before the fix, the dominant loss was **B (KB depth)** — the correct category was chosen, but the knowledge base simply did not contain the obligations a senior QA expects (file-upload rules for Employee, payment-failure matrix for Checkout, the entire reset-token lifecycle for Password Reset).

That is the textbook signal to fix **one** stage: **deepen the knowledge base**. Login proved the mechanism — it already had the deepest KB and it already scored 100%.

5. The fix — one stage, one file

File changed: `src/engines/qa-knowledge-engine.ts` (**knowledge base depth only**). No planner rewrite, no new engine, no classifier change, no new architecture.

Added, each gated on evidence keywords so nothing leaks into unrelated requirements:

- **crud:** whitespace-only rejection, unauthorized-create, searchable-after-create, and a four-case **file-upload** obligation set (valid upload, bad format, oversize, corrupt) gated on `photo/image/upload/attachment/file/document/avatar/picture`.
- **checkout:** a full **payment-failure matrix** (declined / expired / invalid CVV / insufficient funds / bad card format) gated on `card/payment/pay/charge`, plus stock-vs-quantity, inventory decrement, confirmation email, double-submit, and session-timeout.
- **authentication:** eleven **password-reset** obligations (request, token-valid, email-sent, no-enumeration, token-expired, token-invalid, single-use, password mismatch, complexity, rate-limit, old-password-invalidated) gated on reset vocabulary so **plain login is untouched** — which is exactly why Login stayed at 100%.

Each new authentication obligation ships **complete, app-neutral scenario semantics** (variableUnderTest, preconditions, single-variable variation, expectedBehavior, requiredDataRole) so the script-generation contract stays intact.

Diff: additions to one file. `npx tsc --noEmit` is clean.

6. What was deliberately not fixed (the next earliest stages)

Discipline means fixing one stage and reporting the rest honestly rather than chasing every number.

- **Leave Request is misrouted (stage A, classification).** Senior QA expects `workflow`; the classifier picks `crud` on the shared `form/submit` signals. This is why Leave still carries **1 critical gap** (“Insufficient balance rejected”) and sits at 60% — the balance/approval/authorization obligations live in the wrong category to be consulted. **Fixing classification is the next earliest stage; it was left untouched this sprint on purpose.**
- **Default coverage mode is positive-only (stage C).** After the KB fix, the largest remaining bucket (38 items) is validations the KB now has but the positive-only default never emits. That is a coverage-selection decision, downstream of KB depth — a later iteration.
- **A handful of genuinely field-specific mediums** (Employee “leading zero preserved” / “special characters in ID”, Checkout “price change during checkout”, Password Reset “very long password boundary”) are intentionally **not** stuffed into a generic KB. The planner stays field-agnostic; these belong to the LLM write-up stage, which sees the concrete fields. Reported as honest residual gaps, not hidden.

The earliest-loss tally flipped from **KB-depth-dominant** (before) to **coverage-selection-dominant** (after: 38 C / 10 A / 8 B) — proof the loss moved cleanly downstream and the right stage was fixed.

7. Regression proof

- `tests/unit/scenario-planner.test.ts` — **53/53 pass** (includes the contract that every authentication scenario has complete, filename-free semantics; the 11 new reset obligations satisfy it).
 - All five benchmarks re-measured after the fix: **no requirement regressed**; Login held at 100%.
 - Six unrelated suites fail **identically on a clean tree** before this change (`execution-provider`, `test-coverage-engine`, `architecture-contract`, `intelligence-orchestrator`, `intelligence`, `profile-diff-engine`). They are **pre-existing** and were not touched — verified by stashing this change and re-running.
-

8. Conclusion

The existing pipeline was **not** fundamentally broken and did **not** need replacing. It needed its knowledge base deepened at one stage. One file changed; critical gaps dropped from 5 to 1; nothing regressed; and the two remaining weaknesses (Leave classification, positive-only default) are measured, named, and queued as the next earliest stages — not papered over.

Artifacts: `scripts/gold-benchmarks.ts` (sealed), `scripts/test-case-audit.ts` (harness), `scripts/audit-output/*.json` (raw per-stage dumps behind every number above).